

Skills平台完整开发方案

Skills 平台

完整开发方案

基于 SaaS CRM · Skill Gateway · 数字员工体系

本文档包含架构设计、接口定义、数据库表结构、开发任务拆解，可直接用于研发启动。

1. 项目概述

在现有 SaaS CRM 基础上，新建一套 Skills 平台，将 CRM 业务能力（会员洞察、导购话术、沉睡召回、营销策略）封装为标准化 skill，通过统一接口对外提供，支持任意 Agent 框架（OpenAI、Claude、Dify、自研）调用。

1.1 核心目标

- 对外提供标准化 skill 能力，任意 Agent 通过 REST 接口调用
- 数字员工身份体系，scope 与 SaaS 权限服务单一数据源对齐
- 现有 SaaS 系统和业务微服务零改动
- 第一阶段交付：Skill Gateway + 数字员工管理页 + member_summary skill

1.2 改动范围

模块	改动类型	说明
Skill Gateway	全新建设	独立服务，完全新建，独立数据库
数字员工管理页	新增入口	嵌入 SaaS 导航，新增独立菜单
SaaS 用户/权限	只读引用	不改代码，Gateway 实时 gRPC 查询
业务微服务	不改动	Gateway 用 service account 内网直连

2. 技术架构

2.1 整体分层

层级	组件	职责
调用方	外部 Agent	OpenAI / Claude / Dify / 自研，持有 skill_token 调用
接入层	Skill Gateway	身份认证、scope 解析、skill 路由、调用审计
能力层	Skills	member_summary 等具体 skill 实现
数据层	业务微服务	会员/订单/商品/指标/营销，gRPC 内网
权限层	SaaS 权限服务	单一数据源，saas_user_id → store_ids[]

2.2 技术选型

项目	选型	说明
Skill Gateway 语言	Go	并发友好，gRPC 原生支持，启动快
对外协议	HTTPS REST	兼容所有 Agent 框架
内网通信	gRPC	复用业务服务现有 .proto
身份凭证	JWT HS256	有效期 8h，含 de_id + saas_user_id
数据库	PostgreSQL	独立实例，3 张表
scope 缓存	本地内存 TTL 5min	减少对权限服务的压力

3. Skill Gateway 对外接口

3.1 POST /auth/token 换取 skill_token

数字员工凭 client_id 和 client_secret 换取 JWT，有效期 8 小时。Agent 启动时调用一次，token 过期后重新获取。

请求体：

```
{  
  "client_id": "cid_xxxxxxx",  
  "client_secret": "sk_xxxxxxx"  
}
```

响应体：

```
{  
  "access_token": "eyJhbGciOi...",  
  "expires_in": 28800,  
  "de_id": "de_xxxxxxx",  
  "de_name": "营销分析助手"  
}
```

3.2 GET /v1/skills 获取 skill 列表

返回平台所有可用 skill 的描述，格式兼容 OpenAI tools[]，Agent 注册后 LLM 自动决策调用时机。

Header: Authorization: Bearer {access_token}

响应体（OpenAI tools[] 格式）：

```
[[  
  {"type": "function",  
    "function": {  
      "name": "member_summary",  
      "description": "查询门店会员概况，含新增、流失、RFM分层",  
      "parameters": { "date": "string", "limit": "number" }  
    }  
  }  
]]
```

3.3 POST /v1/skills/{skill_id}/invoke 执行 skill

核心调用接口。Gateway 完成鉴权、scope 解析、路由执行后返回结构化数据。

Header: Authorization: Bearer {access_token}

请求体：

```
{ "params": { "date": "2026-03-14", "limit": 100 } }
```

成功响应（200）：

```
{
  "run_id": "run_xxxxxxx",
  "skill_id": "member_summary",
  "latency_ms": 230,
  "data": { ... },
  "scope": "store:store_001"
}
```

3.4 错误码

HTTP	场景	error 字段
401	token 无效 / 过期	unauthorized · token_expired
403	scope 权限不足	forbidden · scope_not_allowed
404	skill_id 不存在	not_found · skill_not_registered
502	业务服务超时/失败	upstream_error · 降级返回空数据
503	权限服务不可达	dependency_unavailable · scope_fetch_failed

4. 数字员工管理

4.1 数据模型

数字员工只存身份信息，不存权限数据。scope 在每次调用时实时从 SaaS 权限服务获取。

字段	类型	说明
id	VARCHAR PK	de_xxxxxxx

name	VARCHAR NOT NULL	数字员工名称
client_id	VARCHAR UNIQUE	换 token 凭证 ID
client_secret	VARCHAR	bcrypt 存储，任何接口不返回原文
saas_user_id	VARCHAR NOT NULL	关联 SaaS 员工，scope 唯一来源
status	VARCHAR	active / disabled
created_by	VARCHAR	创建人 saas_user_id
created_at / updated_at	TIMESTAMP	

4.2 管理接口

接口	方法	说明
POST /de	创建	生成 client_id/secret，secret 明文仅返回一次
GET /de	列表	返回基本信息，不含 secret
GET /de/{id}/scope	权限预览	实时查 SaaS 权限服务，返回 store_ids[]
POST /de/{id}/reset-secret	重置	旧 secret 立即失效，新 secret 明文仅返回一次
PATCH /de/{id}/status	启用/禁用	禁用后存量 token 最长 8h 后自然失效
DELETE /de/{id}	删除	不可恢复，run_log 历史保留

4.3 管理页面

- 入口：SaaS 导航栏 → 系统管理 → 数字员工
- 新建流程：填名称 → 搜索绑定 SaaS 员工 → 确认 → 一次性展示凭证
- 详情页展示实时权限预览（可见门店列表）
- 支持启用/禁用/重置 secret/删除操作

🔒 client_secret 仅在创建和重置时以明文展示，关闭弹窗后不可再查，引导用户立即复制

5. Skill 注册表

5.1 表结构

字段	类型	说明
skill_id	VARCHAR PK	如 member_summary
name	VARCHAR NOT NULL	展示名称
description	TEXT	LLM 读取的功能描述，直接影响调用时机判断
parameters_schema	JSONB	入参定义，生成 OpenAI tool 参数描述
handler	VARCHAR NOT NULL	Go handler 函数名，路由时使用
required_scope_type	VARCHAR	最低 scope: store / group / zone / brand
status	VARCHAR	active / disabled

5.2 第一批 skill

skill_id	名称	scope 要求	状态
member_summary	会员概况查询	store+	✓ 第一阶段实现
guide_script	导购话术生成	store+	后续迭代
dormant_recall	沉睡会员召回	store+	后续迭代
campaign_suggest	营销策略推荐	zone+	后续迭代

6. 数据库完整建表 SQL

Gateway 独立数据库，共三张表。

6.1 digital_employee

```

CREATE TABLE digital_employee (
  id      VARCHAR(32) PRIMARY KEY,
  name    VARCHAR(100) NOT NULL,
  client_id  VARCHAR(64) UNIQUE NOT NULL,
  client_secret VARCHAR(256) NOT NULL, -- bcrypt
  saas_user_id VARCHAR(64) NOT NULL, -- scope 来源
  status   VARCHAR(16) DEFAULT 'active',
  created_by VARCHAR(64),
  created_at  TIMESTAMP  DEFAULT NOW(),
  updated_at  TIMESTAMP  DEFAULT NOW()
);

CREATE INDEX idx_de_client_id  ON digital_employee(client_id);
CREATE INDEX idx_de_saas_user_id ON digital_employee(saas_user_id);
CREATE INDEX idx_de_status     ON digital_employee(status);

```

6.2 skill_registry

```

CREATE TABLE skill_registry (
  skill_id      VARCHAR(64) PRIMARY KEY,
  name          VARCHAR(100) NOT NULL,
  description    TEXT      NOT NULL,
  parameters_schema JSONB,
  handler        VARCHAR(100) NOT NULL,
  required_scope_type VARCHAR(16) DEFAULT 'store',
  status         VARCHAR(16) DEFAULT 'active',
  created_at     TIMESTAMP  DEFAULT NOW(),
  updated_at     TIMESTAMP  DEFAULT NOW()
);

CREATE INDEX idx_sr_status ON skill_registry(status);

```

6.3 run_log

```

CREATE TABLE run_log (

```

```
run_id    VARCHAR(32) PRIMARY KEY,
de_id     VARCHAR(32) NOT NULL, -- 不设外键
skill_id  VARCHAR(64) NOT NULL, -- 不设外键
scope_snapshot VARCHAR(512),    -- 调用时 scope 快照
params    JSONB,
status    VARCHAR(16) NOT NULL, -- success/error
latency_ms INT,
error_msg TEXT,
created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_rl_de_id    ON run_log(de_id);
CREATE INDEX idx_rl_skill_id ON run_log(skill_id);
CREATE INDEX idx_rl_created_at ON run_log(created_at);
CREATE INDEX idx_rl_status  ON run_log(status);
```

6.4 初始化数据

```
INSERT INTO skill_registry VALUES (
    'member_summary',
    '会员概况查询',
    '查询指定门店的会员概况，包含会员总数、新增、流失和 RFM 分层数据',
    '{"date": "string", "limit": "number"}',
    'MemberSummaryHandler',
    'store',
    'active'
);
```

7. 代码结构

skill-gateway/


```
├── cmd/
|   ├── main.go          # 启动入口
├── internal/
|   ├── auth/
|   |   ├── token.go      # JWT 签发与验签
|   |   ├── employee.go   # digital_employee 查询
|   ├── scope/
|   |   ├── resolver.go   # 查 SaaS 权限服务
|   |   ├── cache.go      # 本地缓存 TTL 5min
|   ├── skill/
|   |   ├── router.go     # skill_id → handler 映射
|   |   ├── registry.go   # 注册表 CRUD
|   |   ├── discovery.go  # 生成 OpenAI tools[] 格式
|   ├── skills/
|   |   ├── base_handler.go # Handler 接口定义
|   |   ├── member_summary.go # 第一个 skill 实现
|   ├── infra/
|   |   ├── grpcclient/
|   |   |   ├── member/    # 会员服务 gRPC client
|   |   |   ├── order/     # 订单服务 gRPC client
|   |   |   ├── metric/    # 指标服务 gRPC client
|   |   |   ├── marketing/ # 营销服务 gRPC client
|   |   |   ├── permission/ # SaaS 权限服务 client
|   ├── audit.go          # run_log 异步写入
|   ├── db.go             # 数据库连接池
├── proto/                 # 复用业务服务 .proto
└── config.yaml
```

8. 开发任务拆解

第一周：跑通端到端链路

序	任务	交付物	工作量
1	搭建 Gateway 项目骨架	Go 项目结构 + config + 健康检查接口	0.5d
2	数据库建表	3 张表 + 索引 + 初始化数据	0.5d
3	身份认证模块	POST /auth/token, JWT 签发 + 验签	1d
4	scope 解析模块	查 SaaS 权限服务 + 本地缓存 5min	1d
5	skill 路由 + 注册表	GET /v1/skills + skill_id → handler 映射	1d
6	member_summary skill 实现	并发调用会员/订单/指标服务 + 聚合	1.5d
7	调用审计	run_log 异步写入	0.5d

第一周验收标准：用 curl 模拟 Agent，调用 member_summary skill，拿到真实业务数据。

第二周：数字员工管理页面

序	任务	交付物	工作量
8	数字员工管理接口	POST/GET/PATCH/DELETE /de 系列接口	1.5d
9	SaaS 前端：列表页	数字员工列表 + 状态操作	1d
10	SaaS 前端：新建弹窗	搜索员工 + 创建 + 凭证展示	1d
11	SaaS 前端：详情页	基本信息 + 权限预览 + 重置 secret	1d
12	联调 + 测试	前后端联调，完整流程验证	1.5d

第三周：完善 + 上线准备

序	任务	交付物	工作量
13	错误处理 + 降级	超时/失败降级，标准错误码	1d
14	日志 + 监控	结构化日志，接入现有监控	1d

15	安全加固	限流、secret 轮转、审计完善	1d
16	接入测试 Agent	用真实 Agent（Dify 或自研）完整验证	1d
17	上线部署	Docker 镜像 + K8s 配置 + 上线	1d

9. 关键设计决策

决策点	选择	原因
scope 数据源	SaaS 权限服务实时查询	单一数据源，权限变更自动同步，无维护成本
不经过现有 Gateway	Skill Gateway 独立入口	避免改动现有 SaaS 主链路，降低风险
token 有效期	8 小时	Agent 无需频繁换 token，scope 5min 缓存可接受
secret 安全	bcrypt 存储，明文仅展示一次	参考 AWS AccessKey 模式，最小化泄露风险
run_log 不设外键	逻辑关联，不物理约束	保留历史审计，删除 DE/skill 不级联
gRPC 并发调用	errgroup 并发 + 3s 超时降级	多数据源并发提升响应速度，降级保证可用性
禁用生效策略	status 禁用 + 重置 secret 立即失效	禁用自然过期（≤8h），立即失效用重置 secret

10. 开发注意事项

- 业务微服务需确认现有接口支持 store_ids[] 数组过滤，若只支持单个 store_id 需提前沟通
- SaaS 权限服务需提供 saas_user_id → store_ids[] 的 gRPC 查询方法，提前确认接口可用
- 数字员工管理页面搜索员工下拉，需要 SaaS 提供用户搜索接口（GET /saas/users?q=）

- Gateway 部署需要能访问所有业务微服务内网地址，提前申请 service account 和网络权限
- scope 缓存 TTL 5min，员工权限变更后最长 5min 生效，业务上可接受
- 第一阶段只实现 member_summary，其他 skill 在注册表里预置记录但 status=disabled